ESPRIT – Année Universitaire 2026-2027
Module : Architecture des Applications Web (AWD)

AWD

DOSSIER TECHNIQUE DE CONCEPTION & MODÉLISATION MICROSERVICES

Projet LearnHub – Validation 1 (Architecture Distribuée 6+1)

Classe : 4SSA13 | **Groupe :** G1
Étudiants : Selim Hechmi & Équipe LearnHub
Microservice de Selim Hechmi : Marketplace & Billing (Port 8085) + Identity (Service commun)
Dépôt Git : Public avec enseignant ajouté comme collaborateur

I. Présentation du Projet & Analyse Fonctionnelle

Idée générale : LearnHub est une plateforme web intelligente qui relie, au sein d'un écosystème distribué, la découverte de cours, l'apprentissage structuré en chapitres, l'évaluation certifiante, l'achat en ligne, l'échange communautaire et l'accompagnement personnalisé par Intelligence Artificielle.

Problématique réelle : Les étudiants souffrent de la dispersion de leurs données entre de multiples plateformes isolées (vidéos séparées des quiz, absence de suivi centralisé et manque d'accompagnement interactif). LearnHub centralise tout le parcours dans une architecture hautement résiliente.

II. Découpage en 7 Microservices (Bounded Contexts)

N°	Microservice	Port	Capacité Métier (Responsabilité)	Base Dédiée
0	**identity-service** (Commun)	`8081`	Gestion des comptes, profils, rôles (Étudiant, Formateur, Admin) et JWT	MySQL (`learnhub_identity_db`)

N°	Microservice	Port	Capacité Métier (Responsabilité)	Base Dédiée
1	**course-service**	`8082`	Catalogue officiel, catégories, cours, chapitres et ressources	MySQL (`learnhub_course_db`)
2	**learning-journey-service**	`8083`	Inscriptions aux cours, suivi de la progression et leçons validées	MySQL (`learnhub_journey_db`)
3	**assessment-service**	`8084`	Banques de quiz, questions, passage de tests et certificats numériques	MySQL (`learnhub_assessment_db`)
4	**marketplace-service**	`8085`	**Responsable : Selim Hechmi** (Commandes, paiement sandbox, factures)	MySQL (`learnhub_marketplace_db`)
5	**community-service**	`8086`	Avis vérifiés sur les cours, fils de discussion et badges de réussite	MongoDB (`learnhub_community_db`)
6	**ai-learning-coach-service**	`8087`	**Service Avancé** (Génération de plans d'étude & chatbot IA)	MongoDB (`learnhub_aicoach_db`)

> **Règle d'or AWD respectée :** Une base dédiée par microservice. Aucun microservice ne lit directement les tables d'un autre service. Les entités distantes sont référencées exclusivement par leur identifiant technique (`courseId` , `studentId`).

III. Communications Interservices

Interaction	Mode	Technologie	Justification & Comportement en cas de panne
`Community ➔ Journey`	Synchrone	OpenFeign	Vérifie qu'un étudiant est inscrit avant d'accepter son

Interaction	Mode	Technologie	Justification & Comportement en cas de panne
			avis. Timeout 2s + Réponse 503 contrôlée.
`Journey → Course`	Synchrone	OpenFeign	Vérifie que le cours existe et est publié avant l'inscription.
`Marketplace → Journey`	Asynchrone	RabbitMQ (`PaymentCompleted`)	Découplage total : crée l'inscription après paiement sans bloquer l'acheteur. Idempotent.
`Community → Course`	Asynchrone	RabbitMQ (`ReviewPosted`)	Met à jour la note moyenne affichée dans le catalogue en tâche de fond.
`Assessment → Community`	Asynchrone	RabbitMQ (`CertificateIssued`)	Community attribue un badge d'accomplissement dès l'obtention du diplôme.

IV. Focus Individuel : Marketplace & Billing (Selim Hechmi)

Le microservice **Marketplace & Billing** est sous la responsabilité de **Selim Hechmi**. Il gère la conversion des cours sélectionnés en commande ferme, la gestion des paiements simulés en environnement sandbox et l'édition automatique des factures.

- **Entités JPA :** `Order` , `OrderItem` , `Payment` , `Invoice` .
- **Endpoints REST exposés :**
  - `POST /api/orders` $ightarrow$ Code `201 Created` (Création de commande)
  - `GET /api/orders/{id}` $ightarrow$ Code `200 OK` (Détail de la commande)
  - `POST /api/orders/{id}/payments` $ightarrow$ Code `201 Created` (Paiement simulé & Facturation)
  - `GET /api/invoices/{id}` $ightarrow$ Code `200 OK` (Téléchargement de facture)
- **Événement RabbitMQ publié :** `PaymentCompleted` contenant `eventId` , `orderId` , `studentId` , `courseId` , `amount` , `occurredAt` .

V. Architecture Technique Globale & Dépôt Git

L'ensemble est orchestré via Spring Cloud (API Gateway, Eureka, Config Server) et Docker Compose. Le dépôt Git public contient l'arborescence complète demandée :

```
learnhub-4ssa13-g1/
├── Backend/ (7 projets Spring Boot initialisés)
├── Frontend/ (Application cliente)
├── Documentation/
│   ├── Presentation/ (Diaporama & Guides)
│   ├── Diagrams/ (SVG d'architecture, cas d'utilisation et classes)
│   └── Dossier_Technique/
└── README.md
```